

UNIT: POLYMORPHISM, INHERITANCE & INTERFACES

Learning Objectives

After completing this unit, students should be able to:

- Explain compile-time and runtime polymorphism.
 - Implement method and constructor overloading.
 - Understand and use `this`, `static`, `super`, and `final`.
 - Explain nested and inner classes.
 - Implement different forms of inheritance.
 - Explain method overriding and dynamic method dispatch.
 - Use abstract classes and abstract methods.
 - Understand Java access modifiers.
 - Define and implement interfaces.
 - Differentiate between classes, abstract classes, and interfaces.
-

1. POLYMORPHISM

1.1 Meaning of Polymorphism

The word **polymorphism** comes from two Greek words:

- **Poly** → Many
- **Morph** → Forms

Therefore, polymorphism means "**many forms.**"

In Java, polymorphism allows the **same method name or reference to behave differently in different situations.**

For example:

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Here, the method name is the same:

```
add()
```

but it works with different parameter types.

1.2 Types of Polymorphism

Java mainly supports two types of polymorphism:

1. Compile-Time Polymorphism

Also called:

- Static polymorphism
- Early binding
- Method overloading

Example:

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

The compiler determines which method should be called.

2. Runtime Polymorphism

Also called:

- Dynamic polymorphism
- Late binding
- Method overriding

Example:

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
  
    void sound() {
```

```
        System.out.println("Dog barks");
    }
}
```

If the following statement is used:

```
Animal a = new Dog();
a.sound();
```

Output:

```
Dog barks
```

The method to execute is determined at runtime.

2. METHOD OVERLOADING

2.1 Definition

Method overloading means defining multiple methods with the **same name but different parameter lists** in the same class.

The parameters can differ in:

- Number
- Type
- Order

Example

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Main Method

```
public class Test {

    public static void main(String[] args) {

        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
    }
}
```

```
        System.out.println(c.add(10, 20, 30));

        System.out.println(c.add(10.5, 20.5));
    }
}
```

Output

```
30
60
31.0
```

2.2 Rules of Method Overloading

The following are valid:

Different number of parameters

```
void display(int a)
void display(int a, int b)
```

Different data types

```
void display(int a)
void display(double a)
```

Different order of parameters

```
void display(int a, double b)
void display(double a, int b)
```

2.3 Return Type Alone Cannot Overload a Method

This is **not valid**:

```
int add(int a, int b) {
    return a + b;
}

double add(int a, int b) {
    return a + b;
}
```

The compiler gives an error because the parameter list is identical.

Important: Method overloading cannot be achieved by changing only the return type.

3. CONSTRUCTOR OVERLOADING

3.1 Constructor

A constructor is a special member of a class used to initialize objects.

Characteristics:

- Same name as class.
 - Does not have a return type.
 - Called automatically when an object is created.
 - Can be overloaded.
-

3.2 Constructor Overloading

When a class contains multiple constructors with different parameter lists, it is called **constructor overloading**.

Example

```
class Student {  
  
    int rollNo;  
    String name;  
  
    Student() {  
        rollNo = 0;  
        name = "Unknown";  
    }  
  
    Student(int r) {  
        rollNo = r;  
        name = "Unknown";  
    }  
  
    Student(int r, String n) {  
        rollNo = r;  
        name = n;  
    }  
  
    void display() {  
        System.out.println(rollNo + " " + name);  
    }  
}
```

Main

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
        Student s2 = new Student(101);  
        Student s3 = new Student(102, "Rahul");  
  
        s1.display();  
        s2.display();  
        s3.display();  
    }  
}
```

Output

```
0 Unknown
101 Unknown
102 Rahul
```

4. `this` KEYWORD

4.1 What is `this`?

`this` is a reference variable that refers to the **current object**.

Example:

```
class Student {
    int rollNo;

    Student(int rollNo) {
        this.rollNo = rollNo;
    }
}
```

Here:

`this.rollNo`

refers to the instance variable, while:

`rollNo`

refers to the constructor parameter.

4.2 Uses of `this`

The `this` keyword has several uses.

1. Refer to current object's instance variable

```
class Student {
    int marks;

    Student(int marks) {
        this.marks = marks;
    }
}
```

2. Call current class method

```
class Student {
```

```
void display() {
    System.out.println("Student");
}

void show() {
    this.display();
}
}
```

`this.display()` calls the current object's `display()` method.

3. Call another constructor

This is called **constructor chaining**.

```
class Student {

    Student() {
        this(100);
        System.out.println("Default constructor");
    }

    Student(int marks) {
        System.out.println("Marks: " + marks);
    }
}
```

Output:

```
Marks: 100
Default constructor
```

Important Rule

`this()` must be the **first statement** inside a constructor.

4. Pass current object as an argument

```
class Student {

    void display(Student s) {
        System.out.println("Student object received");
    }

    void show() {
        display(this);
    }
}
```

5. Return current object

```
class Student {
```

```
        Student getStudent() {  
            return this;  
        }  
    }  
}
```

5. `static` KEYWORD

5.1 Meaning

The `static` keyword is used for members that belong to the **class rather than individual objects**.

A static member is shared among all objects of the class.

5.2 Static Variable

Example:

```
class Student {  
  
    int rollNo;  
    String name;  
    static String university = "GSFC University";  
  
    Student(int r, String n) {  
        rollNo = r;  
        name = n;  
    }  
  
    void display() {  
        System.out.println(rollNo + " " + name + " " + university);  
    }  
}
```

The variable:

```
static String university
```

is shared by all students.

5.3 Static Method

A static method belongs to the class.

```
class Calculator {  
  
    static int square(int n) {  
        return n * n;  
    }  
}
```

It can be called without creating an object:

```
System.out.println( Calculator.square(5) );
```

Output:

25

5.4 Important Rules of Static Methods

A static method:

- Can directly access static variables.
- Cannot directly access instance variables.
- Cannot directly use `this`.
- Cannot directly use `super`.

Example:

```
class Test {  
  
    int x = 10;  
    static int y = 20;  
  
    static void display() {  
  
        // System.out.println(x); // Error  
  
        System.out.println(y); // Correct  
    }  
}
```

6. NESTED CLASSES AND INNER CLASSES

Java allows a class to be declared inside another class.

Such a class is called a **nested class**.

General structure:

```
class Outer {  
  
    class Inner {  
  
    }  
}
```

6.1 Types of Nested Classes

Nested classes are broadly divided into:

1. Static nested class
2. Non-static nested class / Inner class

Java also supports:

3. Local inner class
4. Anonymous inner class

7. INNER CLASS

An inner class is a **non-static class defined inside another class**.

Example:

```
class Outer {  
    int x = 10;  
    class Inner {  
        void display() {  
            System.out.println("x = " + x);  
        }  
    }  
}
```

Creating an object:

```
public class Test {  
    public static void main(String[] args) {  
        Outer obj = new Outer();  
        Outer.Inner inner = obj.new Inner();  
        inner.display();  
    }  
}
```

Output:

```
x = 10
```

8. STATIC NESTED CLASS

A static nested class is declared using the `static` keyword.

```
class Outer {  
    static int x = 100;  
    static class Inner {
```

```

        void display() {
            System.out.println(x);
        }
    }
}

```

Object creation:

```

Outer.Inner obj = new Outer.Inner();

obj.display();

```

Unlike a normal inner class, a static nested class does not require an object of the outer class.

Inner Class vs Static Nested Class

Feature	Inner Class	Static Nested Class
<code>static</code> keyword	No	Yes
Requires outer object	Yes	No
Can access outer instance members	Yes	No, not directly
Associated with	Outer object	Outer class

9. INHERITANCE

9.1 Definition

Inheritance is an OOP mechanism in which one class acquires properties and behaviors of another class.

The existing class is called:

- Parent class
- Superclass
- Base class

The new class is called:

- Child class
- Subclass
- Derived class

9.2 Syntax

```

class Parent {

}

```

```
class Child extends Parent {  
  
}
```

Example:

```
class Animal {  
  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

Main:

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.eat();  
        d.bark();  
    }  
}
```

Output:

```
Animal eats  
Dog barks
```

10. TYPES OF INHERITANCE

Java supports the following types through classes:

1. Single inheritance

```
A  
|  
B
```

Example:

```
class A {  
  
}  
  
class B extends A {  
  
}
```

2. Multilevel inheritance

A
|
B
|
C

Example:

```
class Animal {  
}  
  
class Dog extends Animal {  
}  
  
class Puppy extends Dog {  
}
```

3. Hierarchical inheritance



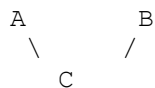
Example:

```
class Animal {  
}  
  
class Dog extends Animal {  
}  
  
class Cat extends Animal {  
}
```

10.1 Multiple Inheritance

Java **does not support multiple inheritance using classes.**

For example:



This is not allowed:

```
class C extends A, B {  
}
```

The reason is ambiguity, commonly illustrated by the **diamond problem**.

However, Java can achieve multiple inheritance of type/behavior through **interfaces**.

Example:

```
interface A {  
    void show();  
}
```

```
}

interface B {
    void display();
}

class C implements A, B {

    public void show() {
        System.out.println("A");
    }

    public void display() {
        System.out.println("B");
    }
}
```

11. CONSTRUCTORS IN INHERITANCE

Constructors are **not inherited** by child classes.

However, when a child object is created, the constructor of the parent class is executed before the child constructor.

Example:

```
class Parent {

    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {

    Child() {
        System.out.println("Child constructor");
    }
}
```

Main:

```
Child c = new Child();
```

Output:

```
Parent constructor
Child constructor
```

11.1 Constructor Execution Order

For:

```
class A
class B extends A
class C extends B
```

Creating:

```
C obj = new C();
```

results in:

```
A constructor  
B constructor  
C constructor
```

Thus, constructor execution occurs from **parent to child**.

12. `super` KEYWORD

`super` refers to the **immediate parent class object/member**.

It is mainly used for:

1. Accessing parent class variable.
 2. Calling parent class method.
 3. Calling parent class constructor.
-

12.1 Access Parent Variable

```
class Parent {  
    int x = 10;  
}  
  
class Child extends Parent {  
    int x = 20;  
  
    void display() {  
        System.out.println(x);  
        System.out.println(super.x);  
    }  
}
```

Output:

```
20  
10
```

12.2 Call Parent Method

```
class Parent {  
    void display() {  
        System.out.println("Parent display");  
    }  
}
```

```

    }
}

class Child extends Parent {

    void display() {

        super.display();

        System.out.println("Child display");
    }
}

```

Output:

```

Parent display
Child display

```

12.3 Call Parent Constructor

```

class Parent {

    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {

    Child() {

        super();

        System.out.println("Child constructor");
    }
}

```

Output:

```

Parent constructor
Child constructor

```

Important Rule

`super()` must be the **first statement** in a constructor.

If no explicit `super()` is written, Java attempts to insert a call to the parent's no-argument constructor automatically.

13. ACCESS MODIFIERS

Access modifiers control the accessibility of classes, variables, methods, and constructors.

Java has four important access levels:

1. public
 2. private
 3. protected
 4. Default / package-private
-

13.1 public

A public member can generally be accessed from anywhere the class itself is accessible.

```
class Student {  
  
    public int marks = 90;  
}
```

Usage:

```
Student s = new Student();  
  
System.out.println(s.marks);
```

14. private

A private member can be accessed **only within the same class**.

```
class Student {  
  
    private int marks = 90;  
  
    void display() {  
        System.out.println(marks);  
    }  
}
```

This is invalid outside the class:

```
Student s = new Student();  
  
System.out.println(s.marks); // Error
```

Private members are commonly used for **encapsulation**.

15. protected

A protected member is accessible:

- Within the same class.
- Within the same package.
- In subclasses, subject to Java's access rules.

Example:

```
class Parent {  
    protected int x = 10;  
}  
  
class Child extends Parent {  
    void display() {  
        System.out.println(x);  
    }  
}
```

16. DEFAULT ACCESS

If no access modifier is specified, the member has **default/package-private access**.

Example:

```
class Student {  
    int marks = 90;  
}
```

The variable can be accessed by classes in the **same package**, but not generally from classes in other packages.

16.1 Access Modifier Comparison

Modifier	Same Class	Same Package	Subclass	Other Package
private	✓	✗	✗	✗
Default	✓	✓ Via package access		✗
protected	✓	✓	✓*	✗*
public	✓	✓	✓	✓

`protected` access in another package has specific rules: the subclass can access the inherited `protected` member through inheritance, with restrictions on how it is referenced.

17. RUNTIME POLYMORPHISM

Runtime polymorphism occurs when a **parent-class reference refers to a child-class object**, and the overridden method that executes is determined at runtime.

Example:

```
class Animal {
```

```
        void sound() {
            System.out.println("Animal sound");
        }
    }

    class Dog extends Animal {

        @Override
        void sound() {
            System.out.println("Dog barks");
        }
    }
}
```

Main:

```
Animal a = new Dog();

a.sound();
```

Output:

```
Dog barks
```

Although the reference type is:

```
Animal
```

the actual object is:

```
Dog
```

Therefore, the overridden `Dog.sound()` executes.

18. METHOD OVERRIDING

18.1 Definition

When a subclass provides its own implementation of a method already defined in its superclass, it is called **method overriding**.

Example:

```
class Animal {

    void sound() {
        System.out.println("Animal sound");
    }
}

class Cat extends Animal {

    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
```

}

18.2 Rules of Method Overriding

Important rules include:

1. Method must have the same name.
 2. Parameter list must be the same.
 3. Return type must be the same or covariant.
 4. The overriding method cannot reduce access.
 5. `final` methods cannot be overridden.
 6. `static` methods are hidden, not overridden.
 7. `private` methods are not overridden.
 8. Constructors cannot be overridden.
-

18.3 Why Use `@Override`?

It is recommended to use:

`@Override`

because it tells the compiler that the method is intended to override a superclass method.

Example:

```
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

If there is a mistake in the method signature, the compiler can identify it.

19. DYNAMIC METHOD DISPATCH

Dynamic method dispatch is the mechanism through which Java determines **at runtime** **which overridden method should be called**.

Example:

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal");  
    }  
}  
  
class Dog extends Animal {
```

```

        @Override
        void sound() {
            System.out.println("Dog");
        }
    }

    class Cat extends Animal {

        @Override
        void sound() {
            System.out.println("Cat");
        }
    }

```

Main:

```

public class Test {

    public static void main(String[] args) {

        Animal a;

        a = new Dog();
        a.sound();

        a = new Cat();
        a.sound();
    }
}

```

Output:

```

Dog
Cat

```

The same reference:

```
Animal a
```

can refer to different objects.

Therefore:

```
a.sound();
```

can produce different results.

20. ABSTRACT CLASSES

An abstract class is a class declared using the `abstract` keyword.

```

abstract class Animal {

}

```

An abstract class can contain:

- Abstract methods
 - Concrete methods
 - Variables
 - Constructors
 - Static members
 - Final members
-

20.1 Abstract Method

An abstract method is declared without a body.

```
abstract class Animal {  
  
    abstract void sound();  
}
```

A subclass must normally implement the abstract method.

```
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

20.2 Abstract Class Example

```
abstract class Shape {  
  
    abstract void area();  
  
    void display() {  
        System.out.println("This is a shape");  
    }  
}  
  
class Circle extends Shape {  
  
    double radius = 5;  
  
    @Override  
    void area() {  
        System.out.println("Area = " + 3.14 * radius * radius);  
    }  
}
```

Main:

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Circle c = new Circle();  
  
        c.display();  
    }  
}
```

```
        c.area();  
    }  
}
```

Output:

```
This is a shape  
Area = 78.5
```

20.3 Can We Create an Object of an Abstract Class?

No.

This is invalid:

```
Shape s = new Shape();
```

because an abstract class cannot be instantiated.

However, an abstract class can have a reference:

```
Shape s = new Circle();
```

This is valid and demonstrates runtime polymorphism.

21. **final** KEYWORD

The `final` keyword is used to restrict modification.

It can be applied to:

- Variables
 - Methods
 - Classes
-

21.1 Final Variable

A final variable cannot be reassigned after initialization.

```
class Test {  
    final int MAX = 100;  
}
```

This is invalid:

```
MAX = 200;
```

21.2 Final Method

A final method cannot be overridden.

```
class Parent {  
  
    final void display() {  
        System.out.println("Parent");  
    }  
}
```

This is invalid:

```
class Child extends Parent {  
  
    void display() {  
        // Error  
    }  
}
```

21.3 Final Class

A final class cannot be inherited.

```
final class Vehicle {  
  
}
```

This is invalid:

```
class Car extends Vehicle {  
  
}
```

Examples of final classes in Java include commonly known classes such as `String`.

21.4 Final Variable vs Static Final Variable

A constant is commonly declared using:

```
static final
```

Example:

```
class MathConstants {  
  
    static final double PI = 3.14159;  
}
```

It is accessed as:

```
System.out.println(MathConstants.PI);
```

22. INTERFACES

22.1 What is an Interface?

An interface defines a **contract** that implementing classes agree to follow.

An interface is declared using:

```
interface
```

Example:

```
interface Animal {  
  
    void sound();  
}
```

A class implements an interface using:

```
implements
```

Example:

```
class Dog implements Animal {  
  
    @Override  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

23. DEFINING AN INTERFACE

Basic syntax:

```
interface InterfaceName {  
  
    // constants  
  
    // abstract methods  
  
}
```

Example:

```
interface Vehicle {  
  
    void start();  
  
    void stop();  
}
```

Implementation:

```
class Car implements Vehicle {
```

```
@Override
public void start() {
    System.out.println("Car starts");
}

@Override
public void stop() {
    System.out.println("Car stops");
}
}
```

24. IMPLEMENTING INTERFACES

A class uses the `implements` keyword.

```
interface Printable {

    void print();
}

class Document implements Printable {

    @Override
    public void print() {
        System.out.println("Printing document");
    }
}
```

Main:

```
public class Test {

    public static void main(String[] args) {

        Document d = new Document();

        d.print();
    }
}
```

Output:

```
Printing document
```

25. MULTIPLE INTERFACES

A class can implement multiple interfaces.

This is one of the ways Java supports **multiple inheritance of type**.

Example:

```
interface Printable {

    void print();
}
```

```

}

interface Showable {

    void show();

}

class Demo implements Printable, Showable {

    @Override
    public void print() {
        System.out.println("Print");
    }

    @Override
    public void show() {
        System.out.println("Show");
    }

}

```

Main:

```

Demo d = new Demo();

d.print();
d.show();

```

Output:

```

Print
Show

```

26. FEATURES OF INTERFACES

Important features include:

1. Interface is declared using `interface`

```

interface Animal {

}

```

2. Classes implement interfaces

```

class Dog implements Animal {

}

```

3. Multiple interfaces can be implemented

```

class Demo implements A, B, C {

}

```

4. Interface methods are public by contract

For traditional abstract interface methods, the implementing method must be `public`.

```

interface A {

```

```
        void show();
    }

    class B implements A {

        public void show() {
        }
    }
}
```

5. Interface fields are implicitly

```
public static final
```

For example:

```
interface Test {

    int MAX = 100;
}
```

is effectively:

```
public static final int MAX = 100;
```

27. DEFAULT METHODS IN INTERFACES

Modern Java interfaces can contain default methods.

Example:

```
interface Vehicle {

    void start();

    default void display() {
        System.out.println("Vehicle");
    }
}
```

A class can use the default implementation:

```
class Car implements Vehicle {

    public void start() {
        System.out.println("Car starts");
    }
}
```

Or override it:

```
class Bike implements Vehicle {

    public void start() {
        System.out.println("Bike starts");
    }
}
```

```
@Override
public void display() {
    System.out.println("Bike");
}
}
```

28. STATIC METHODS IN INTERFACES

Interfaces can also contain static methods.

Example:

```
interface Calculator {

    static int square(int n) {
        return n * n;
    }
}
```

Call:

```
System.out.println(Calculator.square(5));
```

Output:

25

The static method is called using the **interface name**.

29. PRIVATE METHODS IN INTERFACES

Modern Java also allows private methods inside interfaces.

They can be used to share implementation logic between default or static methods.

Example:

```
interface Test {

    default void display() {
        helper();
    }

    private void helper() {
        System.out.println("Helper method");
    }
}
```

The private method is not accessible by implementing classes.

30. INTERFACE REFERENCE

An interface reference can refer to an object of a class that implements the interface.

Example:

```
interface Animal {  
    void sound();  
}  
  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Bark");  
    }  
}
```

We can write:

```
Animal a = new Dog();  
  
a.sound();
```

Output:

Bark

This is another example of **runtime polymorphism**.

31. CLASS VS ABSTRACT CLASS VS INTERFACE

Feature	Class	Abstract Class	Interface
Object creation	Yes	No	No
Can have constructors	Yes	Yes	No
Can have instance variables	Yes	Yes	Not as instance fields
Can have concrete methods	Yes	Yes	Yes, e.g. default/static/private methods
Abstract methods	No requirement	Yes	Traditional interface methods are abstract by default
Multiple inheritance	No through classes	No through classes	Multiple interfaces can be implemented
Keyword	class	abstract class	interface
Inheritance	extends	extends	extends for interfaces

Feature	Class	Abstract Class	Interface
Implementation	—	—	implements
Main purpose	General object modeling	Partial abstraction	Contract/behavior

32. ABSTRACT CLASS VS INTERFACE

This is an important examination topic.

Abstract Class	Interface
Declared using <code>abstract class</code>	Declared using <code>interface</code>
Can have constructors	Cannot have constructors
Can have instance variables	Fields are constants by default
Can have concrete methods	Can have default/static/private methods
Can have abstract methods	Can declare abstract methods
A class extends one class	A class can implement multiple interfaces
Useful for shared state and implementation	Useful for defining a contract
Supports single class inheritance	Supports multiple interface implementation

33. COMPLETE EXAMPLE: INHERITANCE + OVERRIDING + ABSTRACT CLASS

```

abstract class Animal {

    String name;

    Animal(String name) {
        this.name = name;
    }

    abstract void sound();

    void display() {
        System.out.println("Name: " + name);
    }
}

class Dog extends Animal {

    Dog(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

```

```

class Cat extends Animal {

    Cat(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

```

Main:

```

public class Test {

    public static void main(String[] args) {

        Animal a1 = new Dog("Tommy");
        Animal a2 = new Cat("Kitty");

        a1.display();
        a1.sound();

        a2.display();
        a2.sound();
    }
}

```

Output:

```

Name: Tommy
Dog barks
Name: Kitty
Cat meows

```

This single program demonstrates:

- Abstract class
- Constructor
- Inheritance
- super
- Method overriding
- Runtime polymorphism
- Dynamic method dispatch

34. COMPLETE EXAMPLE: INTERFACE + POLYMORPHISM

```

interface Payment {

    void pay(double amount);
}

class CreditCard implements Payment {

```

```

        @Override
        public void pay(double amount) {
            System.out.println("Paid using Credit Card: " + amount);
        }
    }

    class UPI implements Payment {

        @Override
        public void pay(double amount) {
            System.out.println("Paid using UPI: " + amount);
        }
    }
}

```

Main:

```

public class Test {

    public static void main(String[] args) {

        Payment p;

        p = new CreditCard();
        p.pay(5000);

        p = new UPI();
        p.pay(2500);
    }
}

```

Output:

```

Paid using Credit Card: 5000.0
Paid using UPI: 2500.0

```

Here:

```
Payment p;
```

is an interface reference.

The actual object changes:

```
new CreditCard()
```

and

```
new UPI()
```

This demonstrates runtime polymorphism.

35. IMPORTANT DIFFERENCE: OVERLOADING VS OVERRIDING

Method Overloading

Compile-time polymorphism
Usually occurs within same class
Same method name
Parameters must differ
Return type alone cannot distinguish overloaded methods
Compiler decides method
Inheritance not required

Method Overriding

Runtime polymorphism
Requires inheritance
Same method name
Parameters must be same
Return type must be same/covariant
Runtime determines overridden instance method
Inheritance required

Example of Overloading

```
void add(int a, int b)

void add(int a, int b, int c)
```

Example of Overriding

```
class Animal {
    void sound() {}
}

class Dog extends Animal {
    @Override
    void sound() {}
}
```

36. IMPORTANT DIFFERENCE: `this` VS `super`

`this`

Refers to current object
Accesses current class members
`this.x`
`this()` calls another constructor of same class
Can call current class method

`super`

Refers to immediate superclass
Accesses parent class members
`super.x`
`super()` calls parent constructor
Can call parent class method

Example:

```
class Parent {

    int x = 10;
}

class Child extends Parent {

    int x = 20;

    void display() {

        System.out.println(this.x);
        System.out.println(super.x);
    }
}
```

```
}  
}
```

Output:

```
20  
10
```

37. IMPORTANT DIFFERENCE: `this` VS `static`

`this` represents the **current object**, whereas `static` represents a member associated with the **class**.

Example:

```
class Student {  
  
    int rollNo;  
    static String university = "GSFC University";  
  
    Student(int rollNo) {  
        this.rollNo = rollNo;  
    }  
}
```

Here:

```
this.rollNo
```

belongs to an individual object.

Whereas:

```
university
```

is shared among all objects.

38. IMPORTANT DIFFERENCE: `final`, `finally`, `finalize`

These terms are frequently confused.

final

Keyword used for restriction.

```
final int x = 10;
```

finally

Block used with exception handling.

```
try {  
    // code  
}  
finally {  
    // cleanup  
}
```

finalize()

Historically associated with garbage collection, but it is **deprecated and should not be used for modern Java resource management**.

Therefore:

`final`, `finally`, and `finalize()` have different purposes.

39. EXAM-ORIENTED PROGRAM

Consider:

```
class A {  
    int x = 10;  
  
    void display() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
    int x = 20;  
  
    @Override  
    void display() {  
        System.out.println("B");  
    }  
  
    void show() {  
        System.out.println(x);  
        System.out.println(super.x);  
  
        super.display();  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        B obj = new B();  
  
        obj.show();  
  
        A ref = new B();  
  
        ref.display();  
    }  
}
```

Output

```
20
10
A
B
```

Explanation

`x` refers to:

```
B.x
```

Therefore:

```
20
```

`super.x` refers to:

```
A.x
```

Therefore:

```
10
```

`super.display()` explicitly calls the parent method:

```
A
```

But:

```
A ref = new B();
ref.display();
```

uses runtime polymorphism, so the child method executes:

```
B
```

40. QUICK REVISION SUMMARY

Polymorphism

One name, many forms.

Two major types:

```
Polymorphism
├── Compile-time
│   └── Method Overloading
└── Runtime
    └── Method Overriding
```

this

Refers to the **current object**.

Common uses:

```
this.variable  
this.method()  
this()
```

super

Refers to the **immediate parent class**.

Common uses:

```
super.variable  
super.method()  
super()
```

static

Belongs to the **class rather than an individual object**.

Examples:

```
static int count;  
static void display();
```

Inheritance

Allows a child class to acquire features of a parent class.

```
class Child extends Parent
```

Common forms:

- Single
- Multilevel
- Hierarchical

Multiple inheritance is not supported through classes, but multiple interfaces can be implemented.

Access Modifiers

```
private  
default  
protected  
public
```

From most restricted to most accessible:

private → default → protected → public

Runtime Polymorphism

Usually involves:

```
Parent p = new Child();
```

and an overridden method:

```
p.display();
```

The child's overridden method executes.

Abstract Class

```
abstract class Shape {  
    abstract void draw();  
}
```

Cannot be instantiated directly.

final

```
final variable → cannot reassign  
final method  → cannot override  
final class    → cannot inherit
```

Interface

Defines a contract.

```
interface Printable {  
    void print();  
}  
  
class Document implements Printable {  
    public void print() {  
        System.out.println("Printing");  
    }  
}
```

A class can implement multiple interfaces:

```
class Demo implements A, B, C
```

41. MOST IMPORTANT QUESTIONS FOR EXAMINATION

Short-answer questions

1. Define polymorphism.
 2. What is method overloading?
 3. What is constructor overloading?
 4. What is the purpose of `this`?
 5. What is a static variable?
 6. What is an inner class?
 7. Define inheritance.
 8. List the types of inheritance supported by Java classes.
 9. Why doesn't Java support multiple inheritance through classes?
 10. What is the purpose of `super`?
 11. Explain access modifiers.
 12. What is method overriding?
 13. What is dynamic method dispatch?
 14. What is an abstract class?
 15. What is an abstract method?
 16. What is the purpose of `final`?
 17. What is an interface?
 18. What is the difference between `extends` and `implements`?
 19. Can an interface have methods with implementations?
 20. Can an abstract class have a constructor?
-

42. LONG-ANSWER / THEORY QUESTIONS

1. Explain compile-time and runtime polymorphism with suitable examples.
2. Explain method overloading and constructor overloading.
3. Explain the different uses of the `this` keyword.
4. Explain the `static` keyword with suitable examples.
5. Explain nested classes and inner classes.
6. Explain different types of inheritance supported by Java.
7. Explain constructors in inheritance.
8. Explain the three major uses of the `super` keyword.
9. Explain all four Java access modifiers with a comparison table.
10. Explain method overriding and dynamic method dispatch.
11. Explain abstract classes with suitable examples.
12. Explain the different uses of the `final` keyword.
13. Explain interfaces and their features.
14. Explain multiple interface implementation.
15. Differentiate between a class, abstract class, and interface.
16. Differentiate between method overloading and method overriding.
17. Explain runtime polymorphism using an interface.

18. Write a Java program demonstrating inheritance, overriding, `super`, and runtime polymorphism.

43. KEY TAKEAWAYS

The most important concepts from this unit can be remembered as follows:

POLYMORPHISM

↓

Many forms

↓

Overloading + Overriding

INHERITANCE

↓

Reuse parent properties/behaviour

↓

extends

SUPER

↓

Parent class

THIS

↓

Current object

STATIC

↓

Class-level member

ABSTRACT

↓

Incomplete/generalized class or method

FINAL

↓

Restriction

INTERFACE

↓

Contract

↓

implements

One-line definitions for revision

Concept	Definition
Polymorphism	Ability to represent one entity in multiple forms
Overloading	Same method name with different parameters
Overriding	Child provides a new implementation of inherited method
Inheritance	Acquiring properties/behavior from another class
<code>this</code>	Reference to current object
<code>super</code>	Reference to immediate parent class

Concept	Definition
<code>static</code>	Member associated with the class
Inner class	Non-static class defined inside another class
Abstract class	Class that cannot be directly instantiated and may contain abstract methods
Abstract method	Method declared without implementation
<code>final</code> variable	Variable that cannot be reassigned
<code>final</code> method	Method that cannot be overridden
<code>final</code> class	Class that cannot be extended
Interface	A contract implemented by classes
Dynamic dispatch	Runtime selection of an overridden method